

Học máy: Học Không Giám sát và Phân cụm K-means

Quoc Kien

1. Tổng quan về Học Không Giám Sát (Unsupervised Learning)

Trong khi Học có giám sát (Supervised Learning) làm việc với cặp dữ liệu huấn luyện đầy đủ nhãn $\{(\mathbf{x}^{(i)}, y^{(i)})\}$, **Học không giám sát** chỉ nhận đầu vào là một tập hợp các vector đặc trưng không có nhãn mục tiêu kèm theo:

$$\mathcal{D} = \{\mathbf{x}^{(i)}\}_{i=1}^n \quad \text{với } \mathbf{x}^{(i)} \in \mathbb{R}^p$$

Mục tiêu của hệ thống là tự tìm ra các mô thức cấu trúc ẩn (Hidden structures) hoặc quy luật phân bố của dữ liệu. Các nhánh bài toán chính bao gồm: * **Phân cụm (Clustering)**: Gom nhóm các điểm dữ liệu tương đồng vào cùng một cụm. * **Giảm chiều dữ liệu (Dimensionality Reduction)**: Nén không gian thuộc tính từ bậc cao về bậc thấp mà vẫn giữ tối đa lượng thông tin (ví dụ: PCA). * **Phát hiện dị biệt (Anomaly Detection)**: Tìm kiếm các điểm dữ liệu bất thường lệch sâu khỏi phân phối chuẩn.

2. Giải thuật Phân Cụm K-means (K-means Clustering)

K-means là thuật toán phân cụm dựa trên phân hoạch không gian (Partitioning method). Thuật toán cố gắng chia n mẫu dữ liệu thành K cụm khác nhau ($C_1, C_2, \dots, C_K$) cố định sao cho tổng bình phương khoảng cách từ các điểm đến tâm cụm (Centroid) tương ứng là nhỏ nhất.

2.1 Hàm Mục Tiêu Định Lượng (Distortion Function / Cost Function)

Về mặt toán học, K -means tối thiểu hóa hàm chi phí J , còn được gọi là độ méo dạng (Distortion Function):

$$J(c^{(1)}, \dots, c^{(n)}, \mu_1, \dots, \mu_K) = \frac{1}{n} \sum_{i=1}^n \|\mathbf{x}^{(i)} - \mu_{c^{(i)}}\|^2$$

Trong đó: * $c^{(i)} \in \{1, 2, \dots, K\}$ là chỉ số cụm được gán cho mẫu dữ liệu $\mathbf{x}^{(i)}$. * $\mu_k \in \mathbb{R}^p$ là vector tọa độ tâm của cụm thứ k . * $\|\cdot\|$ đại diện cho khoảng cách hình học L2 (Euclidean distance).

Diễn giải cho người mới học. Với mỗi điểm dữ liệu, K -means chỉ hỏi một câu: “Tâm cụm nào gần điểm này nhất?” Khoảng cách bình phương là mức phạt cho phép gán đó. Hàm mục tiêu là mức phạt trung bình trên toàn bộ dữ liệu, nên phân cụm tốt nghĩa là các điểm nằm gần tâm cụm được gán của chúng.

Với một cụm cố định C_k , bước cập nhật tâm cụm giải bài toán nhỏ hơn:

$$\min_{\mu_k} \sum_{i \in C_k} \|\mathbf{x}^{(i)} - \mu_k\|^2$$

Lấy đạo hàm theo μ_k :

$$\sum_{i \in C_k} 2(\mu_k - \mathbf{x}^{(i)}) = 0$$

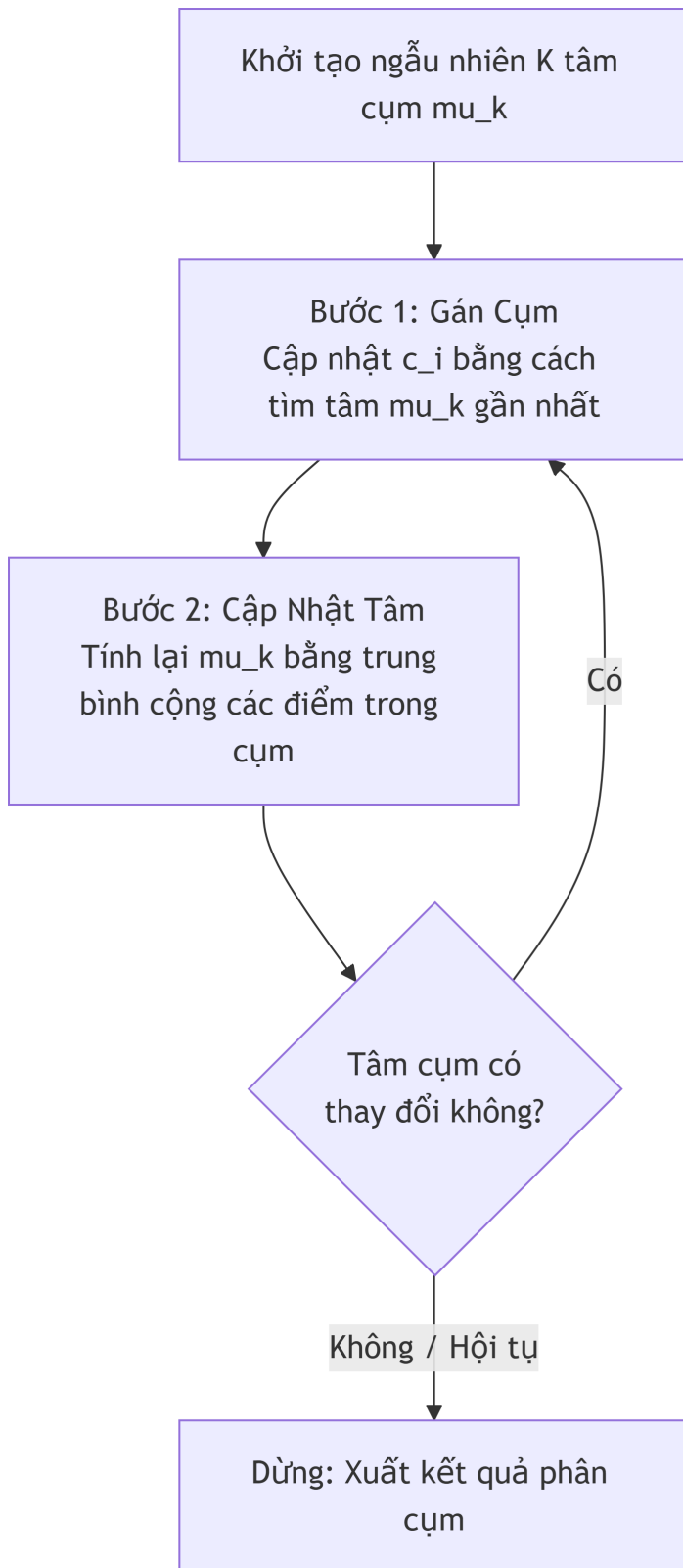
Cô lập tâm cụm:

$$|C_k| \mu_k = \sum_{i \in C_k} \mathbf{x}^{(i)} \implies \mu_k = \frac{1}{|C_k|} \sum_{i \in C_k} \mathbf{x}^{(i)}$$

Vì vậy, tâm cụm tốt nhất cho một nhóm điểm cố định chính là trung bình cộng của các điểm trong nhóm đó.

2.2 Quy Trình Vòng Lặp Hội Tụ (Coordinate Descent)

Hàm mục tiêu J phụ thuộc vào hai nhóm biến độc lập: chỉ số gán cụm $c^{(i)}$ và tọa độ tâm μ_k . Để tối ưu, K -means thực hiện kỹ thuật tối ưu hóa luân phiên theo từng nhóm biến (Coordinate Descent):



1. **Bước gán cụm (Cluster Assignment Step):** Giữ cố định toàn bộ các tâm cụm μ_k , tối ưu hóa hàm chi phí bằng cách tìm chỉ số cụm $c^{(i)}$ gần nhất cho từng mẫu dữ liệu:

$$c^{(i)} \leftarrow \arg \min_k \|\mathbf{x}^{(i)} - \mu_k\|^2$$

2. **Bước cập nhật tâm cụm (Centroid Update Step):** Giữ cố định toàn bộ các chỉ số gán cụm $c^{(i)}$, tối ưu hóa tọa độ các tâm cụm μ_k bằng cách lấy đạo hàm riêng của J theo từng μ_k và đặt bằng 0. Kết quả nghiệm đóng chính là tọa độ trung bình cộng:

$$\mu_k \leftarrow \frac{\sum_{i=1}^n \mathbb{I}(c^{(i)} = k) \mathbf{x}^{(i)}}{\sum_{i=1}^n \mathbb{I}(c^{(i)} = k)} = \frac{1}{|C_k|} \sum_{i \in C_k} \mathbf{x}^{(i)}$$

Chứng minh tính hội tụ: Vì tại mỗi phân đoạn của Bước 1 và Bước 2, giá trị của hàm chi phí J luôn luôn giảm hoặc giữ nguyên ($J^{(t+1)} \leq J^{(t)}$), đồng thời hàm J luôn bị chặn dưới bởi số 0, nên giải thuật K -means **chắc chắn luôn luôn hội tụ** về một điểm cực trị sau một số hữu hạn vòng lặp. Tuy nhiên, vì đây là hàm không lồi hoàn toàn, thuật toán có thể bị kẹt ở điểm cực tiểu cục bộ (Local Minimum) phụ thuộc vào cách khởi tạo tâm cụm ban đầu.

3. Cải Tiến Chọn Tâm Ban Đầu: K-means++

Để giảm thiểu rủi ro bị kẹt ở cực tiểu cục bộ do khởi tạo các tâm ban đầu quá gần nhau, thuật toán **K-means++** bổ sung quy tắc chọn tâm thông minh dựa trên xác suất khoảng cách:

1. Chọn ngẫu nhiên tâm cụm đầu tiên μ_1 từ tập dữ liệu.
2. Với mỗi mẫu dữ liệu $\mathbf{x}^{(i)}$, tính toán khoảng cách ngắn nhất từ nó đến các tâm cụm đã được chọn trước đó, ký hiệu là $D(\mathbf{x}^{(i)})$.
3. Chọn tâm cụm tiếp theo μ_k từ các điểm dữ liệu dựa trên phân phối xác suất tỷ lệ thuận với bình phương khoảng cách:

$$P(\mathbf{x}^{(i)}) = \frac{D(\mathbf{x}^{(i)})^2}{\sum_{j=1}^n D(\mathbf{x}^{(j)})^2}$$

(Điểm nào nằm càng xa các tâm cụm đã có thì càng có xác suất cao được chọn làm tâm cụm mới). 4. Lặp lại bước 2 và 3 cho đến khi đủ K tâm cụm, sau đó tiến hành chạy vòng lặp K -means chuẩn.

4. Thực Hành Trace Thuật Toán Bằng Tay (Bản Số Hóa)

Dưới đây là phần số hóa chi tiết bài toán chạy mô phỏng từng bước (Trace) từ ghi chép thực tế của bạn với 5 mẫu dữ liệu trong không gian 2D ($p = 2$), phân thành $K = 2$ cụm.

4.1 Khởi tạo dữ liệu gốc (Input Data)

- $\mathbf{x}^{(1)} = (1, 1)$
- $\mathbf{x}^{(2)} = (2, 1)$
- $\mathbf{x}^{(3)} = (4, 1)$
- $\mathbf{x}^{(4)} = (7, 1)$
- $\mathbf{x}^{(5)} = (9, 1)$

Chọn 2 tâm cụm khởi tạo ban đầu ngẫu nhiên:

- $\mu_1^{(0)} = (2, 1)$
 - $\mu_2^{(0)} = (4, 1)$
-

4.2 Vòng Lặp 1 (Iteration 1)

Bước 1: Gán cụm dựa trên khoảng cách Euclidean gần nhất

Tính toán khoảng cách từ các điểm dữ liệu đến hai tâm $\mu_1^{(0)}$ và $\mu_2^{(0)}$:

- Mẫu $\mathbf{x}^{(1)} = (1, 1)$: Gần $\mu_1^{(0)}$ hơn $\implies c^{(1)} = 1$.
- Mẫu $\mathbf{x}^{(2)} = (2, 1)$: Trùng với $\mu_1^{(0)} \implies c^{(2)} = 1$.
- Mẫu $\mathbf{x}^{(3)} = (4, 1)$: Trùng với $\mu_2^{(0)} \implies c^{(3)} = 2$.
- Mẫu $\mathbf{x}^{(4)} = (7, 1)$: Gần $\mu_2^{(0)}$ hơn $\implies c^{(4)} = 2$.
- Mẫu $\mathbf{x}^{(5)} = (9, 1)$: Gần $\mu_2^{(0)}$ hơn $\implies c^{(5)} = 2$.

Kết thúc bước gán cụm, ta có thành phần thành viên:

- Cụm 1: $C_1 = \{\mathbf{x}^{(1)}, \mathbf{x}^{(2)}\}$
- Cụm 2: $C_2 = \{\mathbf{x}^{(3)}, \mathbf{x}^{(4)}, \mathbf{x}^{(5)}\}$

Bước 2: Cập nhật lại tọa độ các tâm cụm

Tính toán trung bình cộng tọa độ các điểm thành viên trong mỗi cụm:

- Tâm cụm 1 mới:

$$\mu_1^{(1)} = \frac{(1, 1) + (2, 1)}{2} = (1.5, 1)$$

- Tâm cụm 2 mới:

$$\mu_2^{(1)} = \frac{(4, 1) + (7, 1) + (9, 1)}{3} = \left(\frac{20}{3}, 1\right) \approx (6.67, 1)$$

4.3 Vòng Lặp 2 (Iteration 2)

Bước 1: Gán lại cụm dựa trên các tâm mới $\mu_1^{(1)}$ và $\mu_2^{(1)}$

- Mẫu $\mathbf{x}^{(1)} = (1, 1)$: Khoảng cách đến $\mu_1^{(1)}$ là 0.5, đến $\mu_2^{(1)}$ là 5.17 $\Rightarrow c^{(1)} = 1$.
- Mẫu $\mathbf{x}^{(2)} = (2, 1)$: Khoảng cách đến $\mu_1^{(1)}$ là 0.5, đến $\mu_2^{(1)}$ là 4.67 $\Rightarrow c^{(2)} = 1$.
- Mẫu $\mathbf{x}^{(3)} = (4, 1)$: Khoảng cách đến $\mu_1^{(1)}$ là 2.5, đến $\mu_2^{(1)}$ là 2.67 $\Rightarrow$ Vẫn gần $\mu_1^{(1)}$ hơn!
Điểm $\mathbf{x}^{(3)}$ bị nhảy cụm từ cụm 2 sang cụm 1 $\Rightarrow c^{(3)} = 1$.
- Mẫu $\mathbf{x}^{(4)} = (7, 1)$: Gần $\mu_2^{(1)}$ hơn $\Rightarrow c^{(4)} = 2$.
- Mẫu $\mathbf{x}^{(5)} = (9, 1)$: Gần $\mu_2^{(1)}$ hơn $\Rightarrow c^{(5)} = 2$.

Thành phần thành viên mới sau khi phân định lại:

- Cụm 1: $C_1 = \{\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \mathbf{x}^{(3)}\}$
- Cụm 2: $C_2 = \{\mathbf{x}^{(4)}, \mathbf{x}^{(5)}\}$

Bước 2: Cập nhật lại tọa độ các tâm cụm một lần nữa

- Tâm cụm 1 mới:

$$\mu_1^{(2)} = \frac{(1, 1) + (2, 1) + (4, 1)}{3} = \left(\frac{7}{3}, 1\right) \approx (2.33, 1)$$

- Tâm cụm 2 mới:

$$\mu_2^{(2)} = \frac{(7, 1) + (9, 1)}{2} = (8, 1)$$

4.4 Kết luận hội tụ (Convergence)

Nếu ta tiếp tục chạy sang Vòng lặp 3, việc gán cụm cho các mẫu dựa trên các tâm cụm mới (2.33, 1) và (8, 1) sẽ giữ nguyên kết quả thành viên không đổi so với Vòng lặp 2. Thuật toán chính thức **đạt trạng thái ổn định**: phép gán cụm không đổi và các tâm cụm không còn di chuyển. Với K -means, đây là nghiệm hội tụ cục bộ phụ thuộc vào khởi tạo, không phải bảo đảm nghiệm tối ưu toàn cục cho mọi bộ dữ liệu.

5. Thực Hành Hiện Thực Hóa Thuật Toán Bằng Python

Đoạn mã Python dưới đây lập trình lại toàn bộ quy trình Coordinate Descent của thuật toán K -means chuẩn để bạn chạy kiểm thử:

```
import numpy as np

class KMeansClustering:
    def __init__(self, k=2, max_iters=100):
        self.k = k
        self.max_iters = max_iters
        self.centroids = None

    def fit(self, X):
        # Khởi tạo ngẫu nhiên K tâm cụm từ các điểm dữ liệu thực tế
        n_samples, n_features = X.shape
        random_indices = np.random.choice(n_samples, self.k, replace=False)
        self.centroids = X[random_indices]

        for _ in range(self.max_iters):
            # Bước 1: Gán cụm (Cluster Assignment)
            # Tính khoảng cách từ tất cả các điểm đến toàn bộ K tâm cụm
            distances = np.linalg.norm(X[:, np.newaxis] - self.centroids,
↪ axis=2)
            labels = np.argmin(distances, axis=1)

            # Lưu lại tâm cụm cũ để kiểm tra điều kiện dừng hội tụ
            old_centroids = self.centroids.copy()
```

```

        # Bước 2: Cập nhật tâm cụm (Centroid Update)
        for cluster_idx in range(self.k):
            cluster_points = X[labels == cluster_idx]
            if len(cluster_points) > 0:
                self.centroids[cluster_idx] = np.mean(cluster_points,
                    ↪ axis=0)

        # Nếu tọa độ tâm cụm không đổi, giải thuật đã hội tụ về một
        ↪ nghiệm ổn định
        if np.allclose(old_centroids, self.centroids):
            break

    return labels

# Nạp đúng đồng dữ liệu mẫu từ bài tập trace bằng tay trong ghi chép
X_trace = np.array([
    [1.0, 1.0],
    [2.0, 1.0],
    [4.0, 1.0],
    [7.0, 1.0],
    [9.0, 1.0]
])

# Chạy giải thuật thực nghiệm
np.random.seed(0)
kmeans = KMeansClustering(k=2, max_iters=10)
final_labels = kmeans.fit(X_trace)
print("Tọa độ các tâm cụm hội tụ cuối cùng:\n", kmeans.centroids)
print("Nhãn cụm hội tụ được gán tương ứng cho từng mẫu:", final_labels)

```

Tọa độ các tâm cụm hội tụ cuối cùng:

```

[[8.      1.      ]
 [2.33333333 1.      ]]

```

Nhãn cụm hội tụ được gán tương ứng cho từng mẫu: [1 1 1 0 0]

```

import numpy as np
import matplotlib.pyplot as plt

X_trace = np.array([
    [1.0, 1.0],
    [2.0, 1.0],

```

```

    [4.0, 1.0],
    [7.0, 1.0],
    [9.0, 1.0]
])

centroids = np.array([[2.0, 1.0], [4.0, 1.0]])
history = []

for iteration in range(4):
    distances = np.linalg.norm(X_trace[:, None, :] - centroids[None, :, :],
    ↪ axis=2)
    labels = np.argmin(distances, axis=1)
    distortion = np.mean(np.min(distances, axis=1) ** 2)
    history.append((iteration, centroids.copy(), labels.copy(), distortion))

    new_centroids = centroids.copy()
    for k in range(2):
        if np.any(labels == k):
            new_centroids[k] = X_trace[labels == k].mean(axis=0)
    if np.allclose(new_centroids, centroids):
        break
    centroids = new_centroids

print("Distortion theo từng vòng lặp:", [round(item[3], 3) for item in
    ↪ history])

fig, axes = plt.subplots(1, len(history), figsize=(4 * len(history), 3.2),
    ↪ sharey=True)
if len(history) == 1:
    axes = [axes]

colors = np.array(["#4C78A8", "#F58518"])
for ax, (iteration, centroids_i, labels_i, distortion_i) in zip(axes,
    ↪ history):
    ax.scatter(X_trace[:, 0], X_trace[:, 1], c=colors[labels_i], s=80,
    ↪ edgecolor="black", label="data points")
    ax.scatter(centroids_i[:, 0], centroids_i[:, 1], c=colors, marker="X",
    ↪ s=220, edgecolor="black", label="centroids")
    for point, label in zip(X_trace, labels_i):
        center = centroids_i[label]
        ax.plot([point[0], center[0]], [point[1], center[1]],
    ↪ color=colors[label], alpha=0.55)

```

```

ax.set_title(f"iter {iteration}\nJ={distortion_i:.3f}")
ax.set_xlim(0, 10)
ax.set_ylim(0.4, 1.8)
ax.set_xlabel("$x_1$")
ax.grid(alpha=0.25)
axes[0].set_ylabel("$x_2$")
plt.tight_layout()
plt.show()

```

Distortion theo từng vòng lặp: `[np.float64(7.0), np.float64(2.461), np.float64(1.333)]`

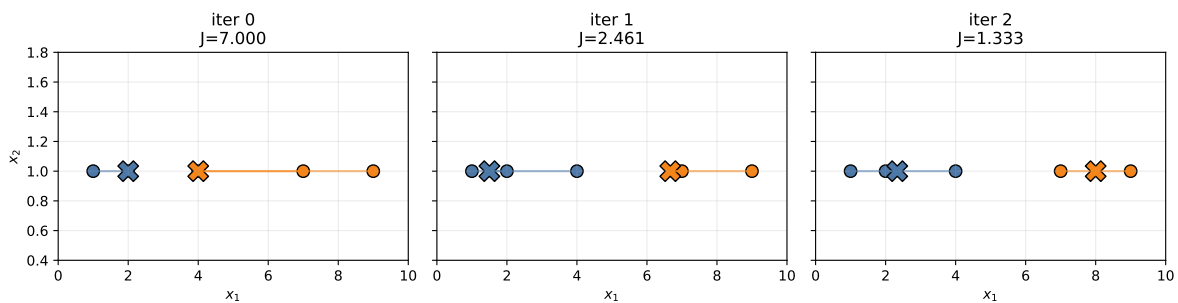


Figure 1: Trace trực quan của K-means: phép gán ổn định dần và distortion giảm đơn điệu.

6. Bảng Công thức Thi: Các Điểm Cốt lõi của K-means

Khái niệm	Điều cần nhớ
Đầu vào	Chỉ có đặc trưng $\mathbf{x}^{(i)}$; không có nhãn $y^{(i)}$.
Mục tiêu	Tối thiểu hóa khoảng cách bình phương trung bình từ mỗi điểm đến tâm cụm được gán.
Bước gán cụm	Chọn tâm gần nhất: $c^{(i)} \leftarrow \arg \min_k \ \mathbf{x}^{(i)} - \mu_k\ ^2$.
Bước cập nhật tâm	Thay mỗi tâm cụm bằng trung bình các điểm được gán vào cụm đó.
Hội tụ	Chi phí không tăng, nhưng nghiệm cuối có thể là cục bộ, không chắc tối ưu toàn cục.
Điểm yếu chính	Nhạy với khởi tạo và giá trị K .

Khái niệm	Điều cần nhớ
Cách cải thiện	Dùng K-means++ và chạy nhiều lần với các khởi tạo khác nhau.

Mẹo nhớ một dòng: bước gán chọn tâm gần nhất; bước cập nhật kéo tâm về trung bình các điểm được gán.

Bộ bài tập mẫu có lời giải

Đề bài. Cho các điểm một chiều $x = \{1, 2, 4, 7, 9\}$ và $K = 2$. Khởi tạo $\mu_1 = 2, \mu_2 = 4$. Hãy thực hiện một vòng K-means đầu tiên.

Lời giải từng bước.

1. Gán cụm theo tâm gần nhất:

- $x = 1$: gần $\mu_1 = 2$ hơn, nên vào cụm 1.
- $x = 2$: trùng $\mu_1 = 2$, nên vào cụm 1.
- $x = 4$: trùng $\mu_2 = 4$, nên vào cụm 2.
- $x = 7$: gần $\mu_2 = 4$ hơn, nên vào cụm 2.
- $x = 9$: gần $\mu_2 = 4$ hơn, nên vào cụm 2.

2. Ta có:

$$C_1 = \{1, 2\}, \quad C_2 = \{4, 7, 9\}$$

3. Cập nhật tâm cụm:

$$\mu_1 = \frac{1+2}{2} = 1.5$$

$$\mu_2 = \frac{4+7+9}{3} = \frac{20}{3} \approx 6.67$$

Kết luận. Sau vòng đầu tiên, hai tâm mới là $\mu_1 = 1.5$ và $\mu_2 \approx 6.67$.

Bài 2: K-means trên bộ dữ liệu 10 điểm hai chiều

Đề bài. Cho 10 điểm:

dòng	x_1	x_2
1	1.0	1.0
2	1.2	0.8
3	0.8	1.3
4	1.5	1.1

dòng	x_1	x_2
5	2.0	1.0
6	7.0	7.0
7	7.5	6.8
8	6.8	7.4
9	8.0	7.2
10	7.2	7.8

Chạy K-means với $K = 2$, khởi tạo $\mu_1 = (1, 1)$ và $\mu_2 = (7, 7)$. Hãy theo dõi nhãn cụm, tâm cụm, và distortion đến khi hội tụ.

Lời giải.

Vòng 0 dùng tâm ban đầu:

$$\mu_1^{(0)} = (1, 1), \quad \mu_2^{(0)} = (7, 7)$$

Ví dụ điểm $(2.0, 1.0)$:

$$d^2((2, 1), \mu_1) = (2 - 1)^2 + (1 - 1)^2 = 1$$

$$d^2((2, 1), \mu_2) = (2 - 7)^2 + (1 - 7)^2 = 61$$

Nên điểm này thuộc cụm 1. Các điểm 1 đến 5 gần μ_1 , các điểm 6 đến 10 gần μ_2 :

$$C_1 = \{(1.0, 1.0), (1.2, 0.8), (0.8, 1.3), (1.5, 1.1), (2.0, 1.0)\}$$

$$C_2 = \{(7.0, 7.0), (7.5, 6.8), (6.8, 7.4), (8.0, 7.2), (7.2, 7.8)\}$$

Distortion vòng 0 là trung bình khoảng cách bình phương đến tâm được gán:

$$J^{(0)} \approx 0.368$$

Cập nhật tâm cụm:

$$\mu_1^{(1)} = \left(\frac{1.0 + 1.2 + 0.8 + 1.5 + 2.0}{5}, \frac{1.0 + 0.8 + 1.3 + 1.1 + 1.0}{5} \right) = (1.30, 1.04)$$

$$\mu_2^{(1)} = \left(\frac{7.0 + 7.5 + 6.8 + 8.0 + 7.2}{5}, \frac{7.0 + 6.8 + 7.4 + 7.2 + 7.8}{5} \right) = (7.30, 7.24)$$

Vòng 1, khi dùng hai tâm mới, nhãn cụm không đổi:

$$C_1 = \text{điểm 1 đến 5}, \quad C_2 = \text{điểm 6 đến 10}$$

Distortion giảm:

$$J^{(1)} \approx 0.248$$

Vì cập nhật lại trung bình vẫn cho (1.30, 1.04) và (7.30, 7.24), thuật toán hội tụ.

Kết luận. Vì dữ liệu tách thành hai nhóm rất rõ, K-means hội tụ nhanh. Với dữ liệu thật, nên chạy nhiều khởi tạo vì kết quả phụ thuộc tâm ban đầu.

Bài 3: Chọn K bằng elbow từ bảng distortion

Đề bài. Giả sử chạy K-means với nhiều K thu được distortion:

K	1	2	3	4	5	6
distortion	38.5	8.2	5.7	4.9	4.5	4.2

Chọn K hợp lý.

Lời giải. Khi tăng K , distortion luôn giảm hoặc giữ nguyên. Ta tìm điểm mà mức giảm bắt đầu chậm lại:

- Từ $K = 1$ sang $K = 2$: giảm 30.3, rất lớn.
- Từ $K = 2$ sang $K = 3$: giảm 2.5.
- Các bước sau giảm rất ít.

Kết luận. Elbow nằm tại $K = 2$. Đây là lựa chọn hợp lý vì thêm cụm sau 2 không cải thiện distortion nhiều.